

ARE 212 SECTION 3:

Functions and looping (feat. OLS)

Fiona Burlig

February 25, 2016

Leftovers

I'll try to start each set of section notes with answers to questions that came up last week, though I won't necessarily go over them in section for the sake of time. So, without further ado:

- Q: *What do chunk labels do in `knitr`?*

A: Not much. I like them as a way of keeping track of what each chunk does - an extra comment, if you will - but they really don't do much else.¹

- Q: *What's up with `bind_rows()`?*

A: The idea behind the `dplyr` package is to have a few functions that efficiently perform basic operations. `mutate()` is designed to work on columns. `bind_rows()` unsurprisingly works to join rows together. The problem we were having last week is that `bind_rows()` works like Stata: if you try to add a vector without column names to a data frame with column names "A", "B", and "C", you'll get the following:

```
A <- 1:5
B <- 2:6
C <- 3:7
myDF<- cbind(A,B,C) %>%
  as.data.frame() %>%
  tbl_df()

myDF

## Source: local data frame [5 x 3]
##
##      A      B      C
##   (int) (int) (int)
## 1     1     2     3
## 2     2     3     4
## 3     3     4     5
## 4     4     5     6
## 5     5     6     7
```

¹You *can* actually call a previous chunk within a chunk - by writing something like: `<<newChunk>>= <<oldChunk>> @`, where `<<oldChunk>>` was defined previously - but don't do this! It's bad and confusing coding practice.

```

D <- t(3:5) %>%
  as.data.frame() %>%
  tbl_df()

problematic <- bind_rows(myDF, D)

problematic

## Source: local data frame [6 x 6]
##
##      A      B      C    V1    V2    V3
##  (int) (int) (int) (int) (int) (int)
## 1     1     2     3    NA    NA    NA
## 2     2     3     4    NA    NA    NA
## 3     3     4     5    NA    NA    NA
## 4     4     5     6    NA    NA    NA
## 5     5     6     7    NA    NA    NA
## 6    NA    NA    NA     3     4     5

```

Eeeew. NA is code for “missing value” in R. Because the entries in C aren’t named, they’re by default given column names V1, V2, and V3 - this means that when we add the D vector to myDF, R adds new columns. To fix this:

```

D <- t(3:5) %>%
  as.data.frame() %>%
  tbl_df()

names(D) <- c("A", "B", "C")

moBettah <- bind_rows(myDF, D)

moBettah

## Source: local data frame [6 x 3]
##
##      A      B      C
##  (int) (int) (int)
## 1     1     2     3
## 2     2     3     4
## 3     3     4     5
## 4     4     5     6
## 5     5     6     7
## 6     3     4     5

```

Beautiful.

Back on track

This section, we'll learn how to build functions and use loops (with a special appearance by parallel programming) - two of the features of R that will be the most helpful for this class and beyond. We'll use our newfound tools to construct an ordinary least squares function from first principles, and then spend most of our time playing with it.

Max has already mentioned to you that using canned routines is prohibited in this class.² Instead, you'll have to build your own tools from the bottom up.

Econometrics junction, what's your function?

This seems like the perfect place to introduce functions. Up to this point, every time we've wanted to manipulate a dataset, we've written a new command. But often, there are commands that we'll want to run over and over again - and many of these commands aren't just single commands, but rather sets of commands - so typing them out a bunch of times is both bad coding practice (overly repetitive) and annoying (overly repetitive).³ In particular, you'll estimate models using Ordinary Least Squares a bajillion⁴ times in this class, and many more throughout your career.

Suppose $\mathbf{y}$ is our outcome variable and $\mathbf{X}$ is a matrix of dependent variables (importantly, including an intercept). As you well know by this point, the linear algebraic formula for OLS is:⁵

$$\beta = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y}$$

Let's code this up in R and test it out using our old friend, the `autos.dta` dataset. Before we do anything else, we'll load our favorite packages, set our working directory, and grab our data:

```
library(haven)
library(dplyr)
```

```
setwd("YOUR SECTION 3 DIRECTORY HERE")
```

```
autos <- read_dta("autos.dta")
```

Note that we have a lot of choices as to how to go about doing this.⁶ I want my OLS function to start from a `tbl_df` object containing my $\mathbf{y}$ variable and my $\mathbf{X}$ variables. Because I'll forget if I have to do it manually every time, I also want my OLS function to automatically add an intercept to my $\mathbf{X}$ variables. Finally, I'd like my OLS function to spit out $\hat{\beta}$.⁷ So, let's code that.

²That being said, I strongly suggest that you check your work using canned routines. I also highly recommend that you keep your eye out for canned routines that do what your home-grown (so green! so Berkeley!) functions will, so that once you move past this course, you have the toolbox to use R in the "real world" of applied econometrics. I'll do my best to point these functions and packages out as we go along.

³Just like those parentheses (overly repetitive).

⁴Technical term.

⁵A friend of mine has this formula tattooed on her knuckles. No joke. Okay, that's not true - but how badass would that be? (Correct answer: not badass at all)

⁶If you don't like my way, too bad! Just kidding. Feel free to write your own version.

⁷Because, duh.

R has really nice function syntax. In particular, the outline of any function you'll ever build will look something like this:

```
OLS <- function(data, y, X) {  
  
}
```

We name the function just like you'd assign a variable name. We tell R that we're calling a function by using the `function()` command, and put the function's inputs, or *arguments*, in parentheses. Then, we code up the steps that the function will perform between the braces. My OLS function will take a dataset, `data`, column name `y` and a vector of column names, `X` as inputs.⁸ Importantly, your function will return (spit out) the last object it sees, unless you tell it otherwise. I like to use the `return()` command in my functions to make sure what I see is what I get.⁹ Let's add that in:

```
OLS <- function(data, y, X) {  
  
  return(betahat)  
}
```

Okay, so that's the basic structure of our function. Now let's make it actually *do* something. The first thing we'd like to do is take our `tbl_df` dataset and grab the right columns. We'd then like to turn these columns into matrices, and to add the intercept to the `X` matrix:

```
OLS <- function(data, y, X) {  
  # grab y from our tbl_df  
  # note the select_() rather than select()  
  # this lets us use strings as inputs  
  ydata <- select_(data, y)  
  # turn y into a matrix (so we can do matrix multiplication)  
  ydata <- as.matrix(ydata)  
  
  # grab the x columns from our tbl_df (again, select_())  
  # This time, we use the .dots input  
  # This allows select_ to use a VECTOR of strings as an input  
  xdata <- select_(data, .dots = X)  
  # now we add a column of ones  
  xdata <- mutate_(xdata, "ones" = 1)  
  # And re-order our variables (intercept, X's)  
  xdata <- select_(xdata, "ones", .dots = X)  
  # turn this whole badboy into a matrix  
  xdata <- as.matrix(xdata)  
  
  return(betahat)
```

⁸There's nothing explicitly in here that says what `data`, `y`, and `X` are. Note, though, that things will break if you try to use the function on the wrong type of object. Real programmers will build in a bunch of error messages to their function - but ain't nobody got time for that in this class.

⁹Just like MS Word! Except way cooler. (That's admittedly a low bar.)

```
}
```

Sweet. Now we've got everything into the right format - we can actually solve for $\hat{\beta}$.

```
OLS <- function(data, y, X) {
  ydata <- select_(data, y)
  ydata <- as.matrix(ydata)

  xdata <- select_(data, .dots = X)
  xdata <- mutate_(xdata, "ones" = 1)
  xdata <- select_(xdata, "ones", .dots = X)
  xdata <- as.matrix(xdata)

  # beta = (X'X)^{-1}X'Y
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  # name it nicely:
  names(betahat) <- c("intercept", X)
  return(betahat)
}
```

Actually, that's way too many annoying intermediate terms. Let's consolidate. To do this, we'll use a cool new tool: the pipe (or chain) command (`%>%`).

This `%>%` command is incredibly useful! It allows us to write functions in a legible way, without having to save intermediate inputs a bunch of times. It takes a dataset on the "left" and applies the function specified on the "right" to it. Need that again? `dataset %>% function`. So, for example:

```
autos %>% select(displacement)

## Source: local data frame [74 x 1]
##
##   displacement
##   (int)
## 1         121
## 2         258
## 3         121
## 4         196
## 5         350
## 6         231
## 7         304
## 8         196
## 9         231
## 10        231
## ..        ...
```

The above code takes our `autos` dataset as an input, and applies the `select(make)` command to it, returning `autos$displacement` (AKA the `displacement` column of the `autos` dataset.) Notice that we don't have to pass

`select()` the `autos` argument anymore. Ahh, sweet efficiency.

Piping might seem a little silly for only one operation, but we can actually chain together a bunch of functions:¹⁰

```
autos %>%
  select(displacement) %>%
  arrange(desc(displacement))

## Source: local data frame [74 x 1]
##
##   displacement
##   (int)
## 1          425
## 2          400
## 3          400
## 4          350
## 5          350
## 6          350
## 7          350
## 8          350
## 9          318
## 10         318
## ..         ...
```

This code chunk returns just the `displacement` column of `autos`, sorted from largest to smallest. We can also save the result to a new variable:

```
sortedDisp <- autos %>%
  select(displacement) %>%
  arrange(desc(displacement))
```

So now let's take our OLS function and clean it up in this same fashion. In particular, we want to eliminate all but one of the of the `ydata <-` and `xdata <-`s.

```
OLS <- function(data, y, X) {
  # grab y, put it into the form we want
  ydata <- data %>%
    select_(y) %>%
    as.matrix()
  # grab x, put it into the form we want
  xdata <- data %>%
    select_(.dots = X) %>%
    mutate_("ones" = 1) %>%
    select_("ones", .dots = X) %>%
    as.matrix()
}
```

¹⁰Note: these have to actually be functions! You can't chain matrix operations. As far as I know, though, anything with a `()` is fair game.

```
# solve for beta hat
betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
# and name it nicely:
names(betahat) <- c("intercept", X)
return(betahat)
}
```

Beautiful! If you don't completely have a handle on `%>%` yet, don't worry - it's not *essential* for anything you'll do in this class, but it will make your life (and mine, as your dear problem set grader) better, so I recommend you practicing with it.

Now that we have our OLS function ready to go, let's take it for a test drive.¹¹ We'll regress price on MPG and weight, and store the output in a variable called `myOLS`.

```
myOLS <- OLS(autos, "price", c("mpg", "weight"))
# and view:
myOLS

##           price
## ones  1946.068668
## mpg   -49.512221
## weight  1.746559
## attr(,"names")
## [1] "intercept" "mpg"      "weight"
```

Okay, that looks plausible: price and MPG are negatively correlated (we saw that last week), and price and weight are positively correlated. But you don't have to take my word for it.¹² Instead, let's verify our results against my favorite canned regression routine. First, we have to install another package, `lfe`.¹³

```
install.packages('lfe')
```

Let's load it into memory:

```
library(lfe)
```

We'll use the `felm()` function, which has the ability to run regressions with lots of (interacted) fixed effects nicely. It also has the ability to run a super simple OLS model, so it's perfect for our purposes:

```
# data = the dataset you want to use
# formula = your regression equation
cannedOLS <- felm(formula = price ~ mpg + weight, data = autos)
```

Notice that we don't have to tell `felm()` to use an intercept - it's smart enough to do that on its own. Nice.¹⁴

Let's compare the canned output with our own:

¹¹Yes, that was another `autos.dta` pun. I'm on a roll.

¹²Yes, that is a Reading Rainbow reference. I am unmistakably a child of the 90's.

¹³Base R will actually also run regressions for you, with the `lm()` function. But `lfe` contains `felm()`, `lm()`'s bigger, badder cousin (whose functionality nests that of `lm()`). Who would rather drive a Datsun than a Porsche? That's what I thought. `felm()` it is.

¹⁴If you want to get `felm()` to run *without* an intercept, set your formula to be `y ~ 0 + x`.

```
myOLS

##           price
## ones  1946.068668
## mpg   -49.512221
## weight 1.746559
## attr(,"names")
## [1] "intercept" "mpg"      "weight"
```

```
cannedOLS

## (Intercept)      mpg      weight
##    1946.069    -49.512      1.747
```

The results are stored slightly differently, but we're getting the same answer. Boom!

If you don't want to have the ease of `dplyr`'s functionality, or just want to run a quick version of OLS on some matrices, the following function is about as bare-bones as you can get and still run OLS.

```
olsMat <- function(y, X) {
  betahat <- solve(t(X) %*% X) %*% t(X) %*% y
  return(betahat)
}
```

Notice that this function does a couple of things differently than our function above: first, its arguments will be matrix objects rather than variable names, and second, this doesn't add an intercept for you - you'll have to do that yourself by `cbind()`ing a column of ones onto your `X` object. Kind of annoying. But, it is quick and dirty. Make sure it works:

```
#this time, we start from a tbl_df and
# convert it into a matrix
ydata <- autos %>% select(price) %>%
  as.matrix()
xdata <- autos %>% select(mpg, weight) %>%
  mutate(ones = 1) %>%
  select(ones, mpg, weight) %>%
  as.matrix()

myOLSMat <- olsMat(ydata, xdata)

myOLS

##           price
## ones  1946.068668
## mpg   -49.512221
## weight 1.746559
## attr(,"names")
## [1] "intercept" "mpg"      "weight"
```



```
myOLSMat

##           price
## ones    1946.068668
## mpg      -49.512221
## weight    1.746559
```

Loops in R or: How I Learned to Stop Worrying and Love apply

Loops are a useful programming tool that allows for the repeated execution of a chunk of code. The most common kind of loop is a `for` loop, which looks something like this:

```
for i = 1/10 {
do something to i
}
```

Notice that this isn't code that will do anything. It actually isn't even R syntax. That's because you shouldn't be using `for` loops (or other loops of this kind) in R. They're slow and inefficient. Instead, we'll use the `apply` family of functions to perform repeated operations in R.¹⁵

Let's take a simple example. First, we'll create a vector containing the numbers 1 through 10. Next, we'll store the square root of each number using `sapply()`.¹⁶

```
myVec <- 1:10
myRoots <- sapply(myVec, function(x) {
  sqrt(x)
})
myRoots

## [1] 1.000000 1.414214 1.732051 2.000000 2.236068 2.449490
## [7] 2.645751 2.828427 3.000000 3.162278
```

Notice what this is doing. `sapply()` takes as inputs a data object and a function. It will “apply” the function to the data object. But it does it in kind of a weird way, using what's called an anonymous function. Rather than defining a new function that will permanently live in our R environment, like we did with `OLS()`, we set up the function within the `sapply()` command. In this case, we tell `sapply()` to evaluate a function. What function? `sqrt()`. What's the input to the function? We call it `x` in `sapply()`'s syntax, but really we're calling the function on each element of our data object.

Let's try a different example. We built this great OLS function, but only tested it once. But our `autos` dataset is many and varied! I'm interested in the unconditional correlations between price and all of the numerical variables

¹⁵Unofficial ARE 212 section motto: no whining!

¹⁶The `apply` family of functions contains a number of different functions, each with slightly different uses. I find `sapply()` to be the most generally useful, and the most useful for the material in this course. Check out Neil Saunders' blog treatment for more info on the other `apply` functions. The `plyr` package has some functions that perform similar operations as well, which are less useful for this class, but will serve you well should you go on to use R regularly in the future.

in our dataset (and without `rep78` because God only knows what that is). To compute these, I could run `OLS()` ten times, or...`sapply()` to the rescue!

First, let's make our lives slightly easier by creating a dataframe that contains only our potential dependent variables, and another with only its variable names.

```
autoNums <- select(autos, -make, -rep78)
autoNames <- names(autoNums)
```

Great. Now I want to run a regression of price on an intercept and each variable in this dataset separately, and store the results:

```
olsOut <- sapply(autoNames, function(x){
  OLS(autoNums, "price", x)
})

olsOut

##               price          mpg headroom      trunk      weight
## intercept 2.728484e-12 11253.0607 4970.3095 3183.5041 -6.707353
## price      1.000000e+00 -238.8943  399.2149  216.7482  2.044063
##               length          turn displacement gear_ratio
## intercept -4584.89902 -2064.9869   3029.04247 12278.028
## price       57.20224   207.5794    15.89588 -2027.544
##               foreign
## intercept 6072.4231
## price      312.2587
```

Boom! Beautiful. But I don't need to see the coefficient on the intercept each time. Let's make a small modification:

```
olsOut2 <- sapply(autoNames, function(x){
  (OLS(autoNums, "price", x))[2]
})

olsOut2

##           price.price          mpg.mpg
##           1.000000          -238.894346
## headroom.headroom      trunk.trunk
##           399.214872          216.748229
## weight.weight          length.length
##           2.044063           57.202238
## turn.turn displacement.displacement
##           207.579425          15.895881
## gear_ratio.gear_ratio foreign.foreign
##          -2027.543860          312.258741
```

The [2] tells R: “only grab the second element of the output of `OLS()`”. Finally, we can compare our results from `OLS()` against the results from `felm()` *en masse*:

```
felmOut <- sapply(autoNums, function(x){
  (felm(data = autoNums, formula = price ~ x))$beta[2]
})
felmOut
```

##	price	mpg	headroom	trunk	weight
##	1.000000	-238.894346	399.214872	216.748229	2.044063
##	length	turn	displacement	gear_ratio	foreign
##	57.202238	207.579425	15.895881	-2027.543860	312.258741

Oops. Clearly `felm()` stores things slightly differently than `OLS()`. Not to worry:

```
felmOut2 <- sapply(autoNums, function(x){
  ((felm(data = autoNums, formula = price ~ x))$beta)[2]
})
felmOut2
```

##	price	mpg	headroom	trunk	weight
##	1.000000	-238.894346	399.214872	216.748229	2.044063
##	length	turn	displacement	gear_ratio	foreign
##	57.202238	207.579425	15.895881	-2027.543860	312.258741

And we test for equality like last week:

```
all.equal(felmOut2, olsOut2, check.names = FALSE)

## [1] TRUE
```

We have a winner!

My [cores] bring all the [econometricians] to the yard

Now that we’re `apply()` ninjas, it’s worth mentioning that R can implement parallel processing. This will make your tasks run much faster (particularly those with thousands of replications), and be a neat trick for parties.¹⁷ Basically, parallel processing is taking advantage of the fact that your computer has more than one CPU or core, and will split tasks among CPUs rather than trying to run them all sequentially on one CPU, thereby dramatically improving speed.

We could use this for a ton of things - basically any time you can imagine iterating over something in R - or better yet, using your newfound `apply()` skills, you can run that in parallel. We’ll explore one neat sidebar here: showing that OLS is unbiased (when things are well-behaved).¹⁸

¹⁷Please do not try to run R processes in parallel at parties. You might crash your friend’s computer! Party foul.

¹⁸Don’t you worry. Things will get badly behaved soon enough.

We'll start by writing a function to generate some fake data, and then run our OLS function on that data:

```
fakeBeta <- function(nreps, sampsi, coeff) {  
  sapply(1:nreps, function(x) {  
    fakeX <- rnorm(sampsi)  
    fakeE <- rnorm(sampsi)  
    fakeY <- coeff * fakeX + fakeE  
  
    fakeMat <- cbind(fakeY, fakeX) %>%  
      as.data.frame() %>%  
      tbl_df()  
    names(fakeMat) <- c("fakeY", "fakeX")  
  
    betahat <- OLS(fakeMat, "fakeY", "fakeX")  
    return(betahat[-1])  
  })  
}
```

Okay, that's cool. I'd like to really show that OLS gets it right in asymptopia, so I want to do a ton of runs with huge sample sizes. Like, I want to do 10,000 runs with sample sizes of 10,000. But that would take forever with one processor. So let's beef things up - parallel style.¹⁹

I mentioned earlier that essentially what parallel processing does is divide up your task into pieces, and send each piece to a different part ("core") of your machine. If you can divide your machine into four parts, it can be running four sub-tasks at once, rather than just one. Sweet. As with most things in R, there are multiple ways to do this. We don't actually need to install the `parallel` package - it's now part of the base R distribution, but we still do need to load it:

```
library(parallel)
```

Next, we need to find out how many CPU-subdivisions, or "cores", we have at our disposal:

```
#reminder: wrapping a call in () will auto-evaluate it  
(myCores <- detectCores())  
  
## [1] 4
```

Cool. I have 4 cores.²⁰ Now what we need to do is tell R that we'd like to work in parallel.

```
cl <- makeCluster(myCores - 1)
```

We're going to modify our `fakeBeta` function slightly, so that it uses the parallel version of `sapply()`, `parSapply()`. Note that all we change are two minor things: first, we add an argument - `clusters` to the function. Next, we modify our `sapply()` call to execute `parSapply()` (which is included in the `parallel` package instead. This is called just like `sapply()`, except it has a new first argument: `clusters` (this is why we had to add an argument at the beginning of our function too).

¹⁹Oooh! Aaaah!

²⁰If you're worried about all of your CPU power / RAM being eaten and you're ~~watching YouTube videos~~ doing other work at the same time, it might be wise to use the number of cores you have on your machine *minus one*.

```

parFakeBeta <- function(nreps, sampsi, coeff, clusters) {
  parSapply(clusters, 1:nreps, function(x) {
    fakeX <- rnorm(sampsi)
    fakeE <- rnorm(sampsi)
    fakeY <- coeff * fakeX + fakeE

    fakeMat <- cbind(fakeY, fakeX) %>%
      as.data.frame() %>%
      tbl_df()
    names(fakeMat) <- c("fakeY", "fakeX")

    betahat <- OLS(fakeMat, "fakeY", "fakeX")
    return(betahat[-1])
  })
}

```

Great. Our parallel function is ready to go.

One quirk. Since we're using user-written functions we need to send those to each cluster we're using, with the `clusterExport()` command:

```

clusterExport(cl, "OLS")
clusterExport(cl, "parFakeBeta")

```

We also need to send `dplyr` to the clusters using `clusterEvalQ()`, since it's used in our `OLS()` function:

```

clusterEvalQ(cl, library(dplyr))

## [[1]]
## [1] "dplyr"      "methods"    "stats"      "graphics"    "grDevices"
## [6] "utils"      "datasets"   "base"
##
## [[2]]
## [1] "dplyr"      "methods"    "stats"      "graphics"    "grDevices"
## [6] "utils"      "datasets"   "base"
##
## [[3]]
## [1] "dplyr"      "methods"    "stats"      "graphics"    "grDevices"
## [6] "utils"      "datasets"   "base"

```

And finally, we want to do the cluster version of setting a seed:

```

clusterSetRNGStream(cl, 12345)

```

With all that setup done, we can finally create 10,000 runs of OLS with randomly generated data. We'll save the results in a new variable called `olsReps`. I want to set $\beta = 5$, such that our data generating process (DGP) is:

$$Y = 5X + \varepsilon$$

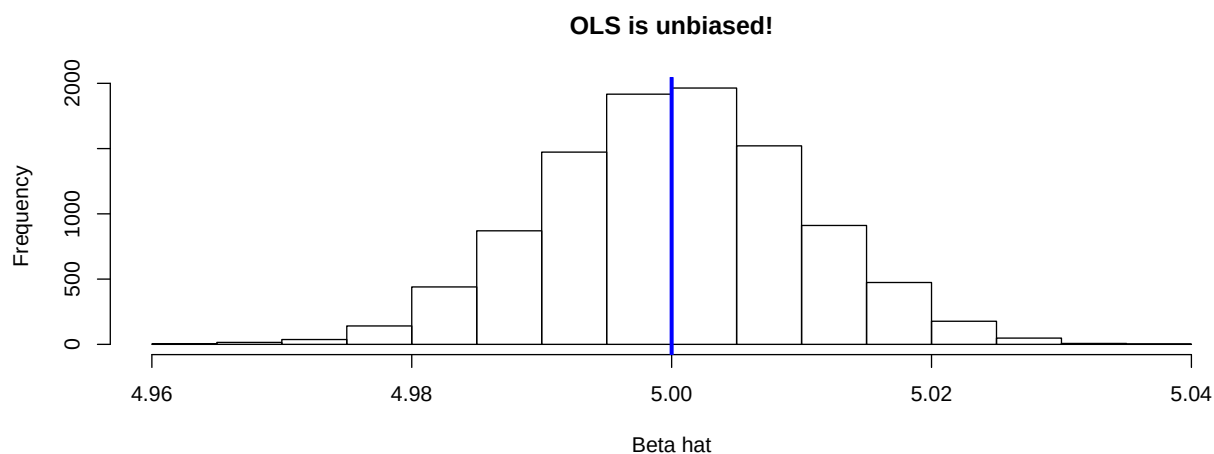
```
olsReps <- parFakeBeta(10000, 10000, 5, cl)
```

Now that we're done with the parallel portion of section, we'll close the clusters:

```
stopCluster(cl)
```

Finally, let's plot a histogram of our simulated $\hat{\beta}$ s, and overlay the true β , 5:

```
hist(olsReps,  
     main="OLS is unbiased!",  
     xlab = "Beta hat")  
# Now add a line of value 5, color blue, and line width 3.  
abline(v=5,col="blue", lwd = 3)
```



OLS. So hot right now.²¹

Extra credit: Bootstrapped standard errors

If you're bored by section and want to try something a little trickier, we can also use `parSapply()` to create bootstrapped standard errors. We won't do regular standard errors for a while - but we can implement the bootstrap version now.

Rather than generate a completely new dataset each run, we'll define a fake dataset like we did above - this time outside of a function, but again with 10,000 observations, and with the following DGP²²:

$$Y = 5X + \varepsilon$$

²¹Can't. Wait.

²²Wouldn't the world be a better place if this were always the DGP? That might put us out of a job, though. Forget I said anything!

```
fakeX <- rnorm(10000)
fakeE <- rnorm(10000)
fakeY <- 5 * fakeX + fakeE

fakeData <- cbind(fakeY, fakeX) %>%
  as.data.frame() %>%
  tbl_df()
names(fakeData) <- c("fakeY", "fakeX")
```

This time, each iteration of our function will create a bootstrapped dataset by drawing rows from our original `fakeData` dataset with replacement, and then run our `OLS()` function on this new dataset. `dplyr` has a convenient built-in function to do this for us: `sample_frac()`. Go easy on your computer to begin with - start with 500 runs. Don't forget to set the seed!

```
set.seed(12345)
# do 500 runs of the stuff inside the {}:
resultsBoot <- sapply(1:500, function(x){
  #creates datasetBoot from rows of autos, with replacement
  datasetBoot <- sample_frac(fakeData, size = 1, replace = TRUE)
  # runs our OLS function on datasetBoot
  # the [-1] keeps only the coeffs. on mpg & weight
  betahatBoot <- (OLS(datasetBoot, "fakeY", "fakeX"))[-1]
  return(betahatBoot)
})
```

Now let's make this generic (rather than specific to `fakeData`), and embed the entire thing into a bootstrapped OLS function that we can call:

```
# we now set the dataset, y, X, and number of replications here
bootOLS <- function(data, y, X, nreps) {
  # this inside bit is the same as before, only genericized
  # shut up, that's a word
  resultsBoot <- sapply(1:nreps, function(x){
    datasetBoot <- sample_frac(data, size = 1, replace = TRUE)
    betahatBoot <- (OLS(datasetBoot, y, X))[-1]
    return(betahatBoot)
  })
}
```

I promised you standard errors, which requires one more step. The bootstrapped standard errors can be expressed as:

$$\hat{s}e_{boot} = \sqrt{\frac{\sum_{b=1}^B (\hat{\beta}_{boot}^b - \frac{1}{B} \sum_{b=1}^B \hat{\beta}_{boot}^b)^2}{B-1}} = \sqrt{\frac{\sum_{b=1}^B (\hat{\beta}_{boot}^b - \hat{\hat{\beta}}_{boot})^2}{B-1}}$$

We can turn this into code pretty easily. Notice what this function does. First, we do the same bootstrapping and OLS procedure as before. Next, we take our bootstrapped $\hat{\beta}$ s and put them into a nicely formatted dataframe.

Finally, we calculate bootstrapped standard errors according to the formula above by first calculating the mean $\hat{\beta}$ across runs for each variable, and then using this as an input into the full-blown bootstrapped standard error formula. This is a much more complicated function than we've written in the past. If what it's doing is confusing to you, go ahead and copy and paste it into your section .R script for now, and examine it more closely at home.

```
bootSE <- function(data, y, X, nreps){
  #start with our bootstrapped results
  resultsBoot <- sapply(1:nreps, function(x){
    datasetBoot <- sample_frac(data, size = 1, replace = TRUE)
    betahatBoot <- (OLS(datasetBoot, y, X))
    return(betahatBoot)
  })
  # to get BS SE's:
  #make resultsBoot into a data table
  resultsBootDF <- resultsBoot %>%
    # transposes the data from being 2 rows into 2 columns
    t() %>%
    # turns it into a dataframe and then a tbl_df
    as.data.frame() %>%
    tbl_df()
  # names the columns
  names(resultsBootDF) <- c("intercept", X)

  #calculate the bootstrapped SE's:
  #first calculate mean(beta boots)

  #Note: this is a fxn over X, which in our example
  # is equal to c("fakeX")

  betaMeans <- sapply(names(resultsBootDF), function(z) {
    # step 1: we want to take the mean over something
    # summarize_() reads strings in a silly way -
    # the entire command has to be in a string, like this:
    # summarize_(data, "mean_(varname)")
    # so we'll create a variable called concat which
    # = "mean(fakeX)" (we do sapply so that we could accomodate
    # more X vars in the future with the same fxn)
    concat <- paste("mean(", z, ")", sep = "")
    # now we can finally call summarize_() on it
    resultsBootDF %>% summarize_(concat)
  })
  # next, for each bootstrapped observation:
  # (Notice: this is called over all the cols of resultsBootDF)
  bootSEs <- sapply(1:dim(resultsBootDF)[2], function(z){
    # calculate (betaBS - meanbetaBS)^2 for each run
    # note again, this function is iterating over VARIABLE NAMES
    # but the command on the next line works on the entire resultsBootDF[,z]
```



```

    # vector all at once
    squaredErrors <- (resultsBootDF[,z] - betaMeans[z])^2
    # next, sum over the squaredErrors vector
    sse <- sum(squaredErrors)
    # finally, actually calculate the bootstrapped SE:
    sqrt(sse / (nreps - 1))
  })
  return(bootSEs)
}

```

Excellent. But 500 runs was boring. I want to do way more runs than that. How about 10,000? Sounds good in theory - but it'll take *forever*.

We closed the cores earlier, so now we need to open them again.

```
cl <- makeCluster(myCores - 1)
```

As above, modify our bootSE() function to make it run in parallel:

```

parBootSE <- function(data, y, X, nreps, clusters){
  #start with our bootstrapped results
  resultsBoot <- parSapply(clusters, 1:nreps, function(x){
    datasetBoot <- sample_frac(data, size = 1, replace = TRUE)
    betahatBoot <- (OLS(datasetBoot, y, X))
    return(betahatBoot)
  })
  # to get BS SE's:
  #make resultsBoot into a data table
  resultsBootDF <- resultsBoot %>%
    # transposes the data from being 2 rows into 2 columns
    t() %>%
    # turns it into a dataframe and then a tbl_df
    as.data.frame() %>%
    tbl_df()
  # names the columns
  names(resultsBootDF) <- c("intercept", X)

  #calculate the bootstrapped SE's:
  #first calculate mean(beta boots)

  #Note: this is a fn over X, which in our example
  # is equal to c("mpg", "weight")

  betaMeans <- sapply(names(resultsBootDF), function(z) {
    # step 1: we want to take the mean over something
    # summarize_() reads strings in a silly way -
    # the entire command has to be in a string, like this:

```

```

#   summarize_(data, "mean_(varname)")
#   so we'll create a variable called concat which
#   = "mean(mpg)" in the first run and "mean(weight)" in the second
concat <- paste("mean(", z, ")", sep = "")
# now we can finally call summarize_() on it
resultsBootDF %>% summarize_(concat)
})
# next, for each bootstrapped observation:
# (Notice: this is called over all the cols of resultsBootDF)
bootSEs <- sapply(1:dim(resultsBootDF)[2], function(z){
  # calculate (betaBS - meanbetaBS)^2 for each run
  # note again, this function is iterating over VARIABLE NAMES
  # but the command on the next line works
  # on the entire resultsBootDF[,z]
  # vector all at once
  squaredErrors <- (resultsBootDF[,z] - betaMeans[z])^2
  # next, sum over the squaredErrors vector
  sse <- sum(squaredErrors)
  # finally, actually calculate the bootstrapped SE:
  sqrt(sse / (nreps - 1))
})
return(bootSEs)
}

```

Great. Our parallel function is ready to go.

Again, we need to send our user-written commands to each cluster we're using, with the `clusterExport()` command:

```

clusterExport(cl, "OLS")
clusterExport(cl, "parBootSE")
clusterExport(cl, "fakeData")

```

We also need to send `dplyr` to the clusters using `clusterEvalQ()`:

```

clusterEvalQ(cl, library(dplyr))

## [[1]]
## [1] "dplyr"      "methods"    "stats"      "graphics"   "grDevices"
## [6] "utils"      "datasets"    "base"
##
## [[2]]
## [1] "dplyr"      "methods"    "stats"      "graphics"   "grDevices"
## [6] "utils"      "datasets"    "base"
##
## [[3]]
## [1] "dplyr"      "methods"    "stats"      "graphics"   "grDevices"
## [6] "utils"      "datasets"    "base"

```

And finally, we want to do the cluster version of setting a seed:

```
clusterSetRNGStream(cl, 12345)
```

With all that setup done, we can finally calculate our bootstrapped standard errors. We'll save the results in a new variable called `bootRepsSE`.

```
bootRepsSE <- (parBootSE(fakeData, "fakeY", "fakeX", 10000, cl))[2]
```

Finally, let's calculate standard errors using the `feIm()` routine:

```
modelSummary <- fakeData %>%  
  feIm(formula = fakeY ~ fakeX) %>%  
  summary()  
cannedSE <- modelSummary$coefficients[-1,2]
```

We've estimated:

```
#bootstrapped SE's  
bootRepsSE  
  
## [1] 0.01003957  
  
# canned SE's  
cannedSE  
  
## [1] 0.009990374
```

Amazing. Close the clusters - job well done.

```
stopCluster(cl)
```